



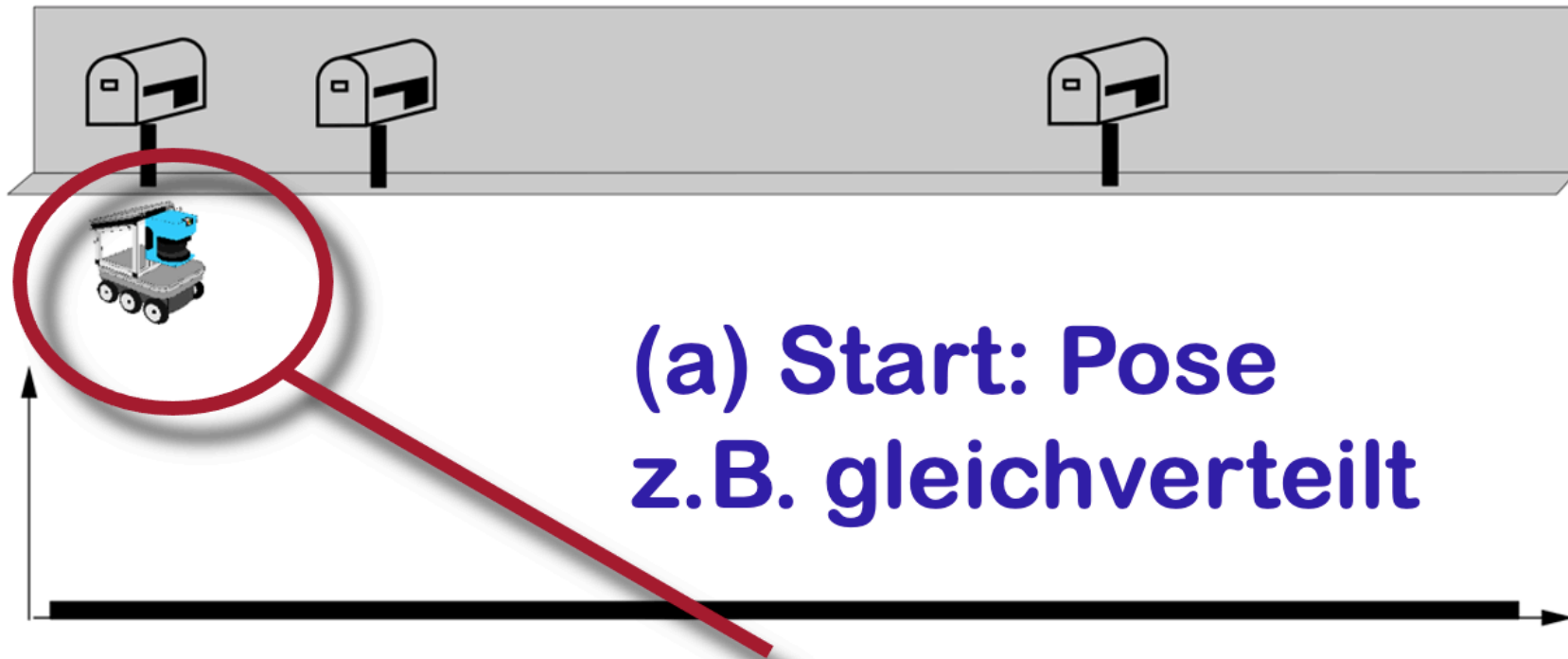
<https://evasys.hs-fulda.de/evasys/online.php?p=6TANR>



Wdh.: Spezieller Bayes-Filter (Idee)

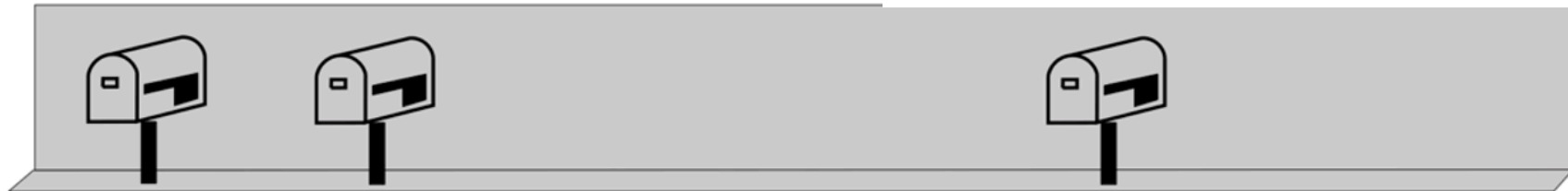
- Voraussetzung: Roboterhandeln + Umgebungsdynamik wird beschrieben als Folge von abwechselnd Aktion / Ereignis und Zustandsmessung, gegeben Start-Zustandsschätzung (Poseverteilung)

1. Führe Aktion aus
2. Schätze resultierenden Zustand für Aktion (a-priori.Zustandsmessung)
3.
$$\overline{Bel}(\mathbf{X}_{t+1}) = \sum_{X_t} P(\mathbf{X}_{t+1} | \mathbf{u}_{t+1}, \mathbf{X}_t) \cdot Bel(\mathbf{X}_t)$$
4. Miss resultierenden Zustand
5. Filterung: berechne aktuelle Zustandsverteilung, die maximal gut zu allen Messungen und der Vorhersage passt (a-posteriori-Zustandsschätzung). Weiter bei 1.
6.
$$Bel(\mathbf{X}_{t+1}) = \eta P(\mathbf{Z}_{t+1} | \mathbf{X}_{t+1}) \cdot \overline{Bel}(\mathbf{X}_{t+1})$$



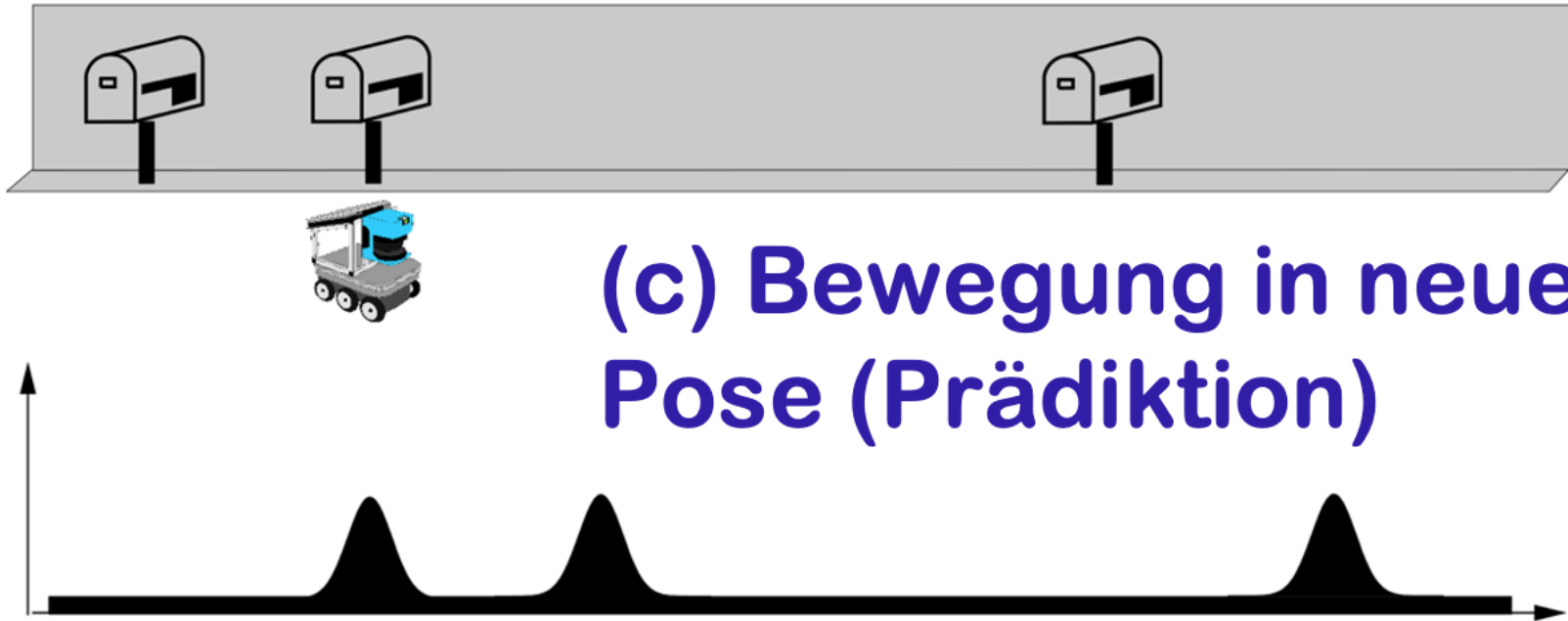
**(a) Start: Pose
z.B. gleichverteilt**

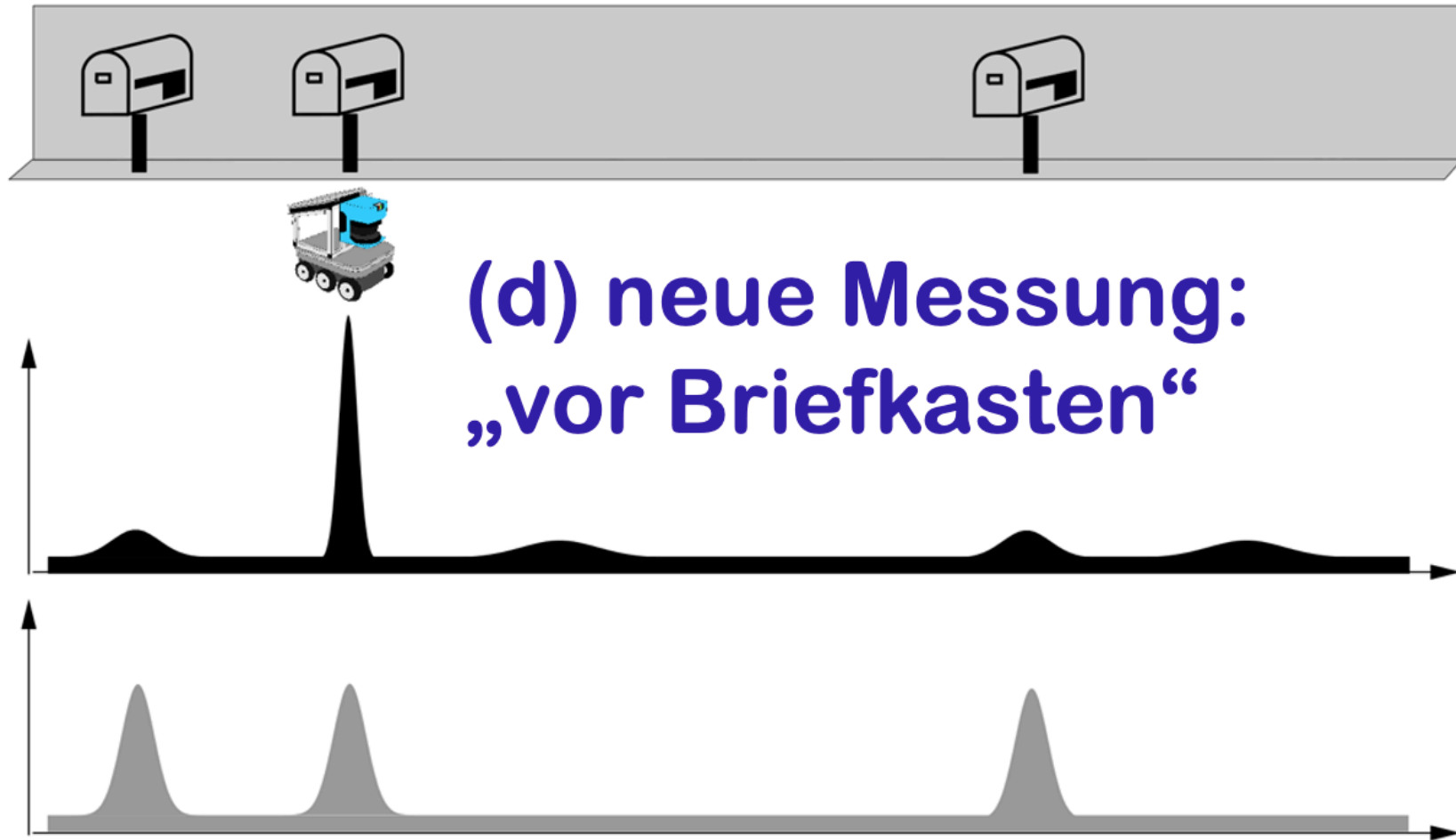
**„objektiv wahre“ Pose,
dem Roboter selber unbekannt!**



**(b) Messung/Filterung:
„vor Briefkasten“**









Naive Markow-Lokalisierung ist naiv

- ▶ Umgebungen interessanter Größe ergeben Verteilungen von nicht handhabbarer Größe!
- ▶ (20m x 50m bei 10 cm und 3.6° Auflösung $\Rightarrow 10^8$ Posen!)
- ▶ Auflösung verringern verwäscht die Lokalisierung
- ▶ $\Rightarrow$ So geht es nur für kleine Umgebungen!

Warum soll man eigentlich unsichere Information bis ins letzte Detail präzise durchrechnen?

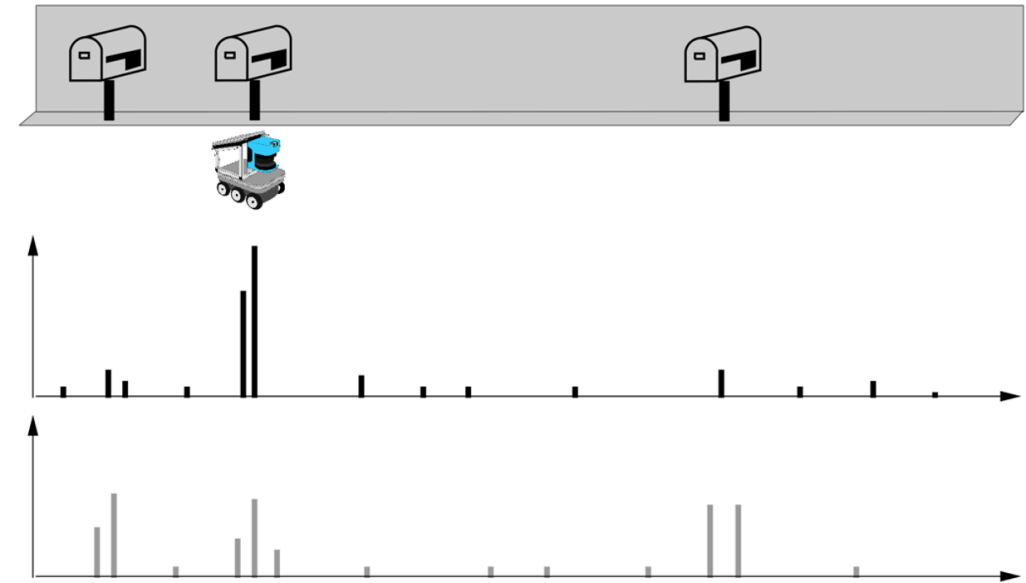
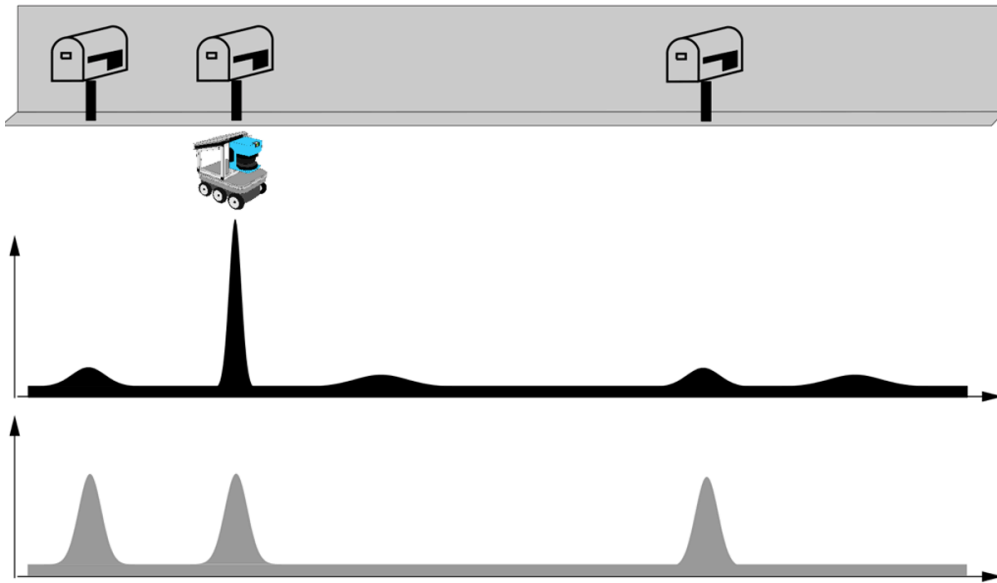


Gliederung

Lokalisierung

Lokalisierung in Karten

1. Arten von Roboterkarten
2. Lokalisierung an Landmarken
3. Markow-Lokalisierung
- 4. Monte-Carlo-Lokalisierung**



Monte-Carlo-Algorithmen approximieren aufwändige exakte Lösungen durch effizientes Berechnen von (relativ wenigen) Stichproben



Rejection Sampling

- ▶ Annahme: Funktion $f(x) > 0$ für alle x
- ▶ Sample x aus einer Gleichverteilung
- ▶ Sample eine Stichprobe c aus $[0,1]$
- ▶ Falls $f(x) > c$ wird die Stichprobe behalten, ansonsten verworfen

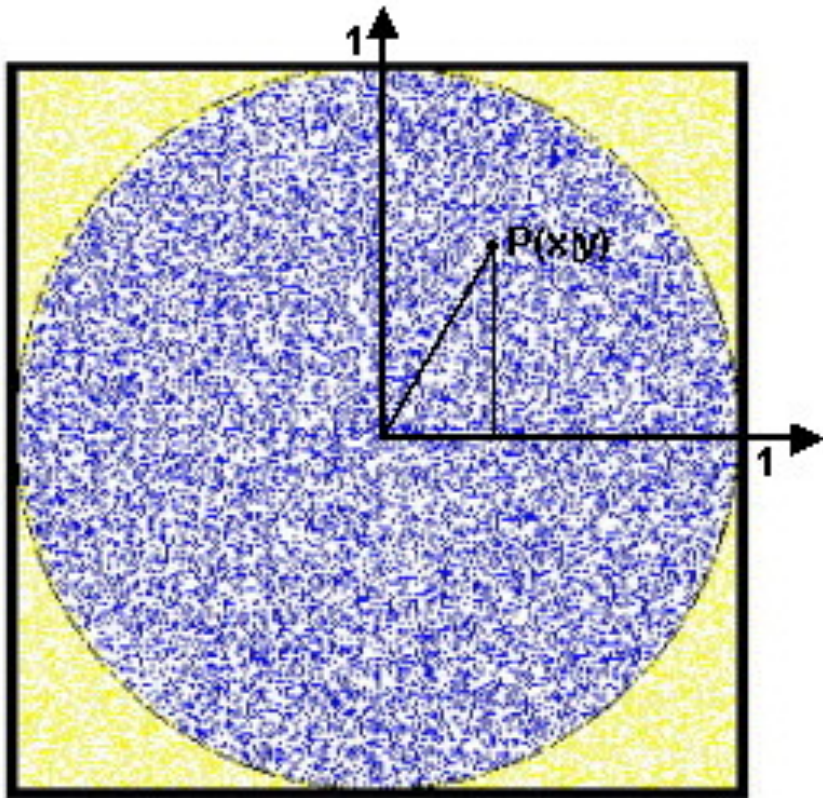
```
/* Take samples and calculate Pi */
for(i = 0; i < number_of_samples; i++){
    /* Get a random point */
    x = random();
    y = random();

    /* Check point and increase counter */
    in_circle(x, y) ? number_inside++ : number_outside++;
}

/* Calculate the value of Pi */
pi = 4.0 * number_inside / number_of_samples;
```



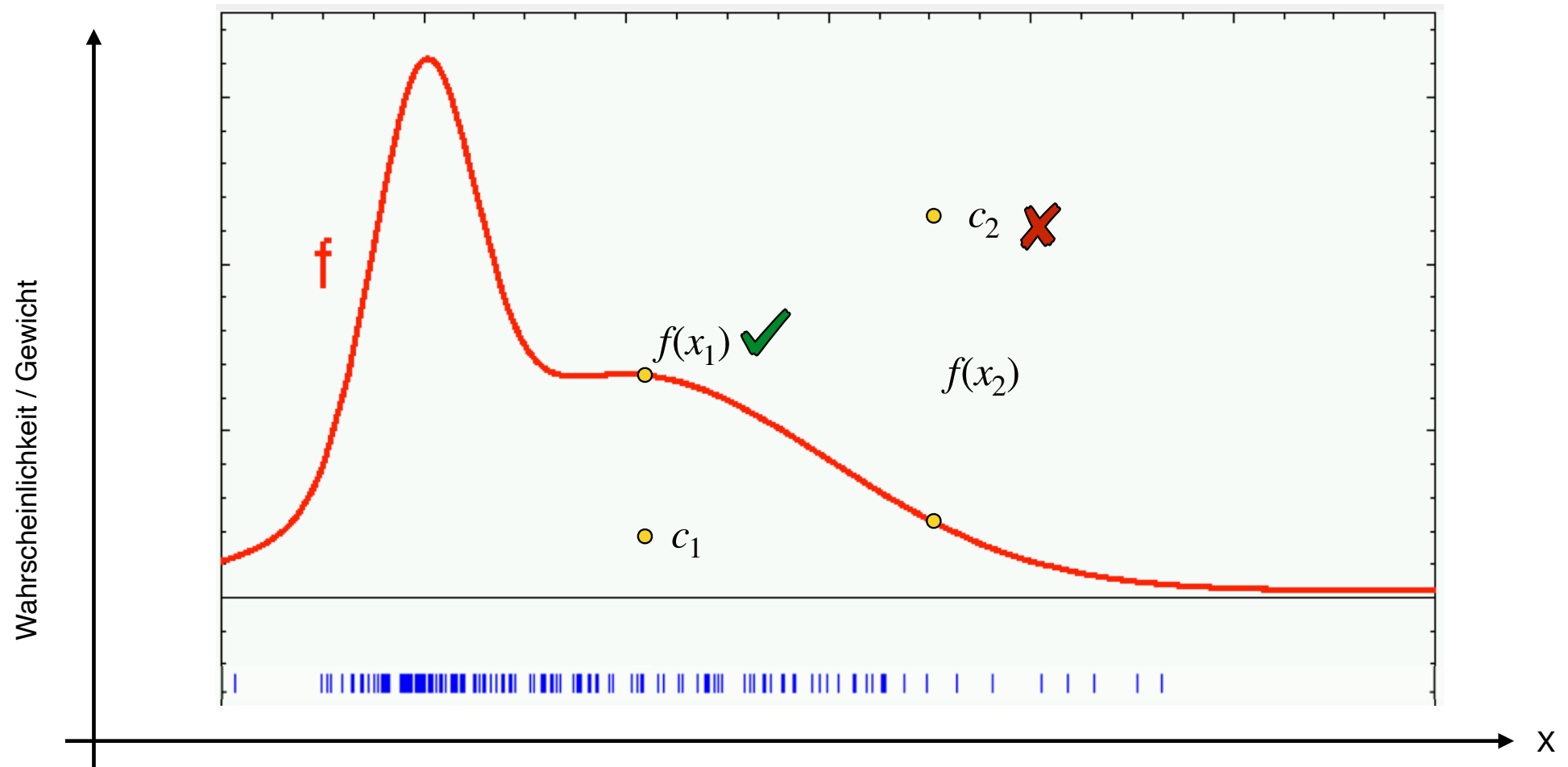

► Monte-Carlo-Berechnung von π



- Ein Kreis mit dem Radius 1 hat den Flächeninhalt π .
- Um den Kreis wird ein Quadrat mit der Seitenlänge 2 gelegt.
- Im Quadrat mit dem Inkreis werden Punkte zufällig gesetzt und dann die Trefferquote berechnet.
- Das Verhältnis aus Trefferzahl (Anzahl der Punkte im Innern des Kreises) und Anzahl der Versuche ist gleich dem Verhältnis aus dem Flächeninhalt des Kreises und dem Flächeninhalt des Quadrates, d.h. die Trefferquote ist gleich $\pi/4$.



Beispiel Rejection Sampling



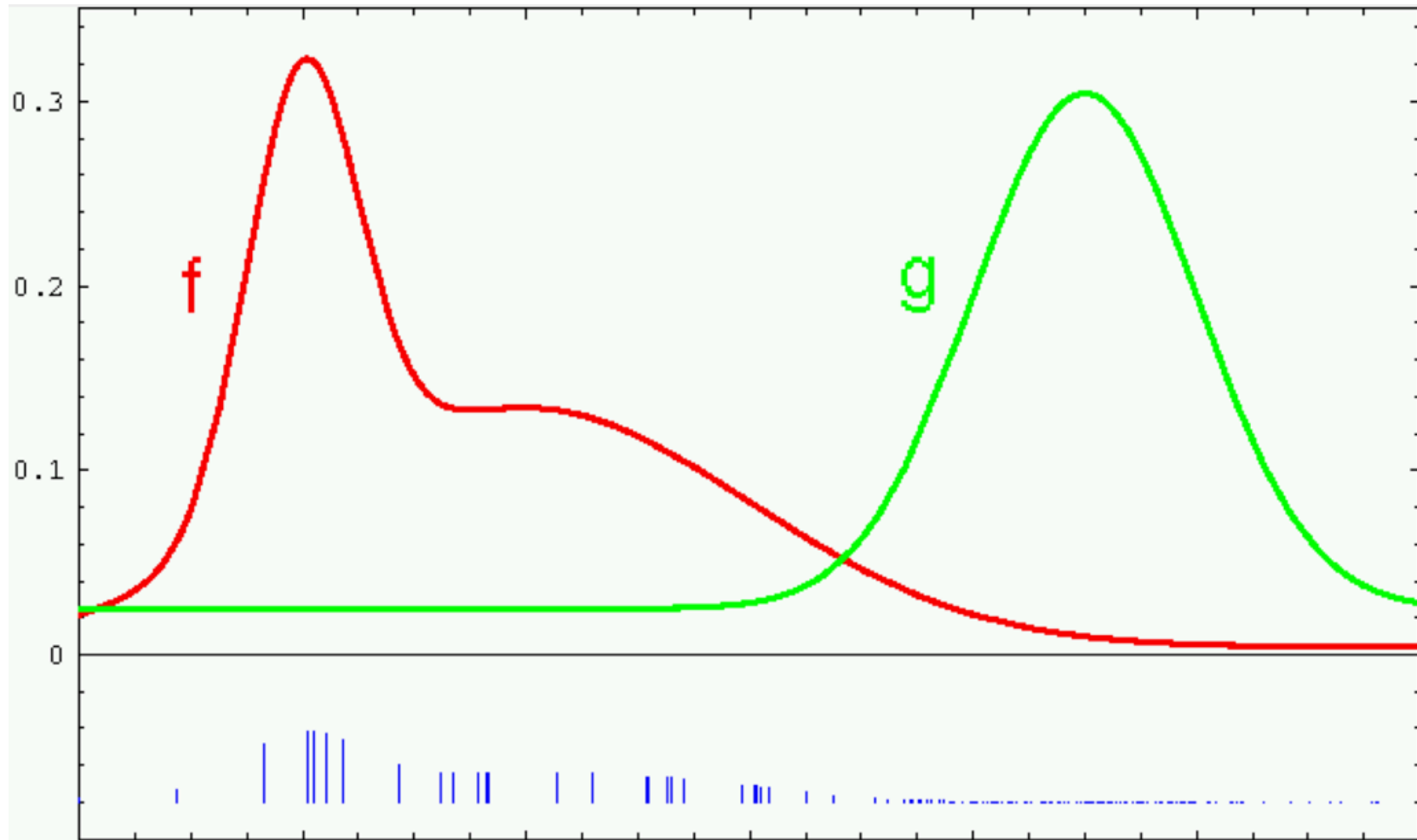


Importance Sampling

- ▶ Man kann auch andere Verteilungen g zum Samplen benutzen
- ▶ Durch ein “Importance Weight” w wird der Unterschied zwischen f und g berücksichtigt
- ▶ $w = f/g$
- ▶ f wird oft als “Target” bezeichnet
- ▶ g wird oft als “Proposal” bezeichnet
- ▶ Vorbedingung: $f(x) > 0 \rightarrow g(x) > 0$



Beispiel: Importance Sampling





Algorithmus MCL

Eingabe : Partikelmenge $\mathcal{X}_t = \{\langle \mathbf{x}_t^i, w_t^i \rangle\}_{i=1, \dots, N}$, Aktion $\mathbf{u}_{t+1}$, Messung z_{t+1} .

Ausgabe: Neue Menge von Partikeln $\mathcal{X}_{t+1}$

```

1:  $\mathcal{X}_{t+1} = \emptyset$ 
2:  $\eta = 0$ 
3: for  $i = 1 \dots N$  do
4:   Ziehe ein  $j$  entsprechend der Verteilung über  $w_t$            //  $Bel(\mathbf{x}_t)$ 
5:   Ziehe ein  $\mathbf{x}_{t+1}^i$  entsprechend  $P(\mathbf{x}_{t+1} | \mathbf{u}_{t+1}, \mathbf{x}_t^j)$  // Transition
6:    $w_{t+1}^i = P(z_{t+1} | \mathbf{x}_{t+1}^i)$                                // Sensormodell
7:    $\eta = \eta + w_{t+1}^i$                                            // Update des Normierungsfaktors
8:    $\mathcal{X}_{t+1} = \mathcal{X}_{t+1} \cup \{\langle \mathbf{x}_{t+1}^i, w_{t+1}^i \rangle\}$ 
9: end for
10: for  $i = 1 \dots N$  do                                       // Normierung der Gewichte in  $\mathcal{X}_{t+1}$ 
11:    $w_{t+1}^i = \eta^{-1} w_{t+1}^i$ 
12: end for
13: return  $\mathcal{X}_{t+1}$ 

```

Gewicht des Partikels

Wie ziehe ich gemäß Verteilung?

Wie definiere ich die Wahrscheinlichkeit einer konkreten Messung



Eingabe : Partikelmenge $\mathcal{X}_t = \{\langle \mathbf{x}_t^i, w_t^i \rangle\}_{i=1, \dots, N}$, Aktion $\mathbf{u}_{t+1}$, Messung z_{t+1} .

Ausgabe: Neue Menge von Partikeln $\mathcal{X}_{t+1}$

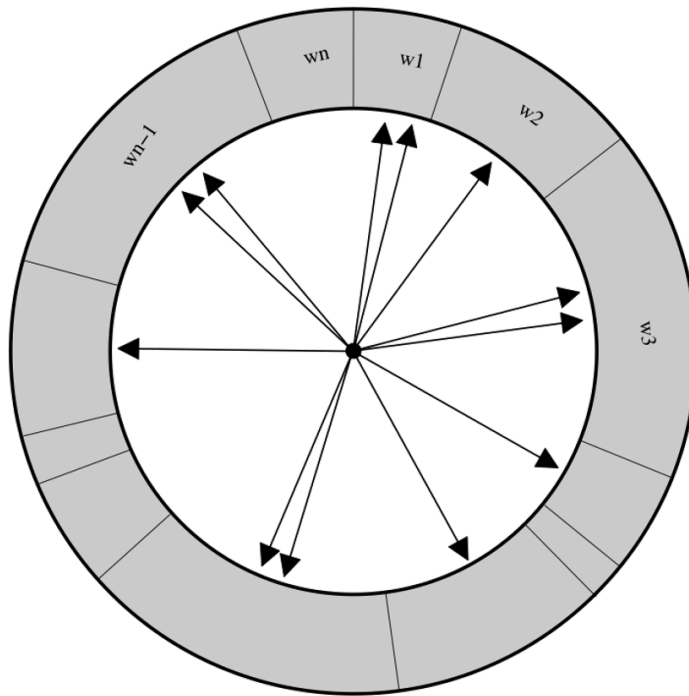
```
1:  $\mathcal{X}_{t+1} = \emptyset$ 
2:  $\eta = 0$ 
3: for  $i = 1 \dots N$  do
4:   Ziehe ein  $j$  entsprechend der Verteilung über  $w_t$  //  $Bel(\mathbf{x}_t)$ 
5:   Ziehe ein  $\mathbf{x}_{t+1}^i$  entsprechend  $P(\mathbf{x}_{t+1} | \mathbf{u}_{t+1}, \mathbf{x}_t^j)$  // Transition
6:    $w_{t+1}^i = P(z_{t+1} | \mathbf{x}_{t+1}^i)$  // Sensormodell
7:    $\eta = \eta + w_{t+1}^i$  // Update des Normierungsfaktors
8:    $\mathcal{X}_{t+1} = \mathcal{X}_{t+1} \cup \{\langle \mathbf{x}_{t+1}^i, w_{t+1}^i \rangle\}$ 
9: end for
10: for  $i = 1 \dots N$  do // Normierung der Gewichte in  $\mathcal{X}_{t+1}$ 
11:    $w_{t+1}^i = \eta^{-1} w_{t+1}^i$ 
12: end for
13: return  $\mathcal{X}_{t+1}$ 
```



Ziehen gemäß Verteilung

- Hier konkret für Partikel-Verteilung, durch Gewichte w_i gegeben, z.B. **Rouletterad-Auswahl**
- Voraussetzung: Ziehen nach Gleichverteilung (approx.: $\text{random}(0, \sum w_i)$)
- Für gezogene Zahl r gilt dasjenige Partikel n als gezogen, für das gilt:

$$\sum_{i=1}^{n-1} w_i < r \leq \sum_{i=1}^n w_i$$



Input: Distribution $\mathcal{X}_t = \{\langle x^i, w^i \rangle\}_{i=1}^N$
Output: Distribution $\mathcal{X}_{t+1}$ with the weights w^i

```

2:  $r_{\max} = \sum_{i=1}^N w^i$ 
3: for  $i = 1 \dots N$  do
4:    $r = \text{random}(0, r_{\max})$ 
5:    $k = 0$ 
6:    $j = 0$ 
7:   while  $k < r$  do
8:      $j = j + 1$ 
9:      $k = k + w^j$ 
10:  end while
11:   $\mathcal{X}_{t+1} = \mathcal{X}_{t+1} \cup \{\langle x^j, w^j \rangle\}$ 
12: end for
13: return  $\mathcal{X}_{t+1}$ 

```



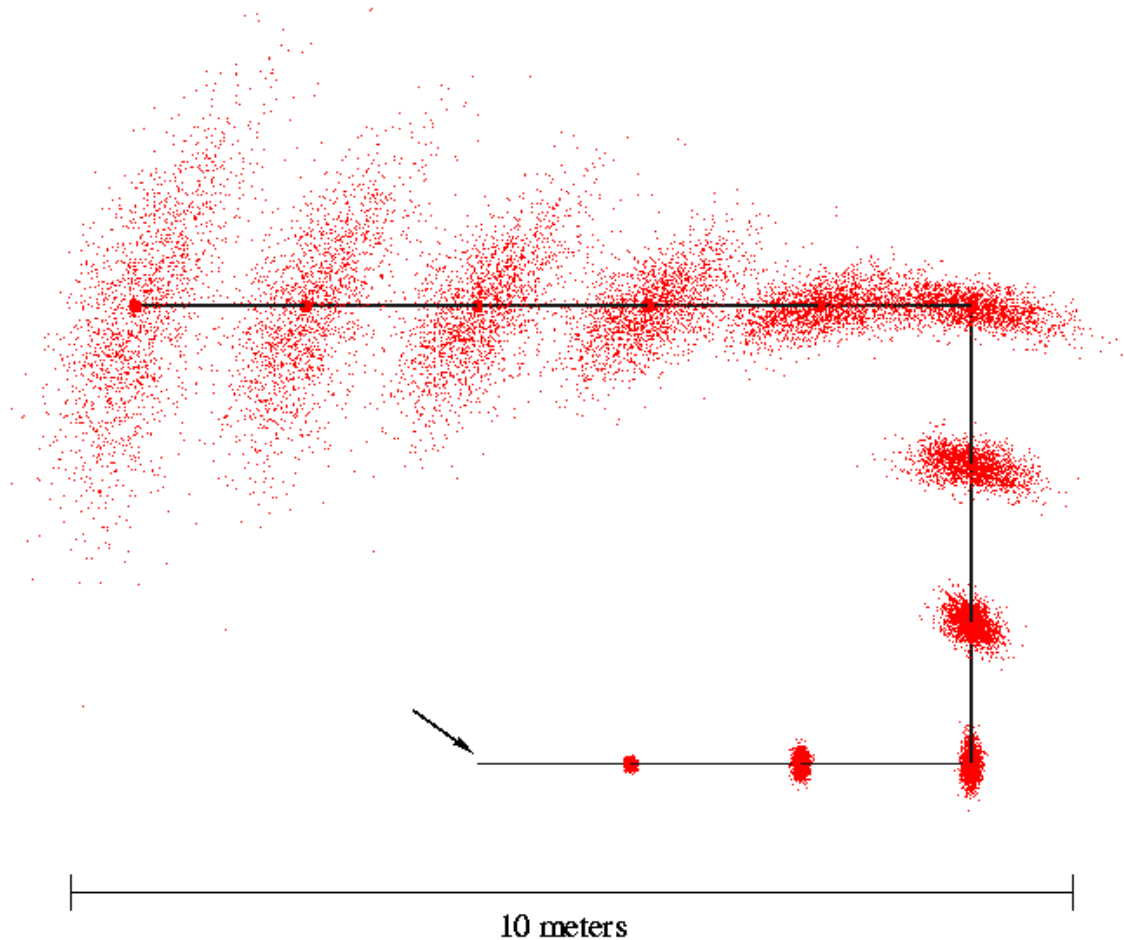
Eingabe : Partikelmenge $\mathcal{X}_t = \{\langle \mathbf{x}_t^i, w_t^i \rangle\}_{i=1, \dots, N}$, Aktion $\mathbf{u}_{t+1}$, Messung z_{t+1} .

Ausgabe: Neue Menge von Partikeln $\mathcal{X}_{t+1}$

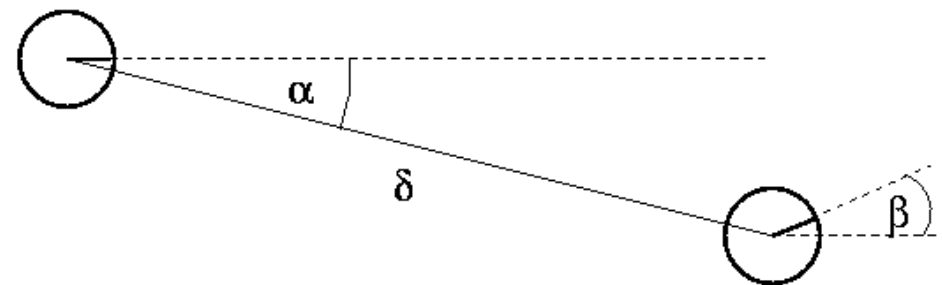
```
1:  $\mathcal{X}_{t+1} = \emptyset$ 
2:  $\eta = 0$ 
3: for  $i = 1 \dots N$  do
4:   Ziehe ein  $j$  entsprechend der Verteilung über  $w_t$  // Bel( $\mathbf{x}_t$ )
5:   Ziehe ein  $\mathbf{x}_{t+1}^i$  entsprechend  $P(\mathbf{x}_{t+1} | \mathbf{u}_{t+1}, \mathbf{x}_t^j)$  // Transition
6:    $w_{t+1}^i = P(z_{t+1} | \mathbf{x}_{t+1}^i)$  // Sensormodell
7:    $\eta = \eta + w_{t+1}^i$  // Update des Normierungsfaktors
8:    $\mathcal{X}_{t+1} = \mathcal{X}_{t+1} \cup \{\langle \mathbf{x}_{t+1}^i, w_{t+1}^i \rangle\}$ 
9: end for
10: for  $i = 1 \dots N$  do // Normierung der Gewichte in  $\mathcal{X}_{t+1}$ 
11:    $w_{t+1}^i = \eta^{-1} w_{t+1}^i$ 
12: end for
13: return  $\mathcal{X}_{t+1}$ 
```



MCL - Bewegungsmodell



Annahme: Odometriefehler ist
Gaußverteilt in α , β und δ





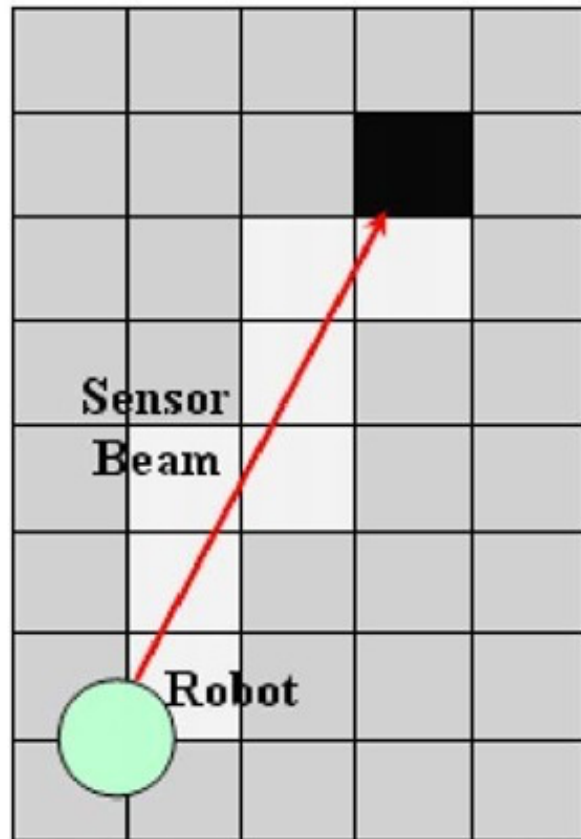
Eingabe : Partikelmenge $\mathcal{X}_t = \{\langle \mathbf{x}_t^i, w_t^i \rangle\}_{i=1, \dots, N}$, Aktion $\mathbf{u}_{t+1}$, Messung z_{t+1} .

Ausgabe: Neue Menge von Partikeln $\mathcal{X}_{t+1}$

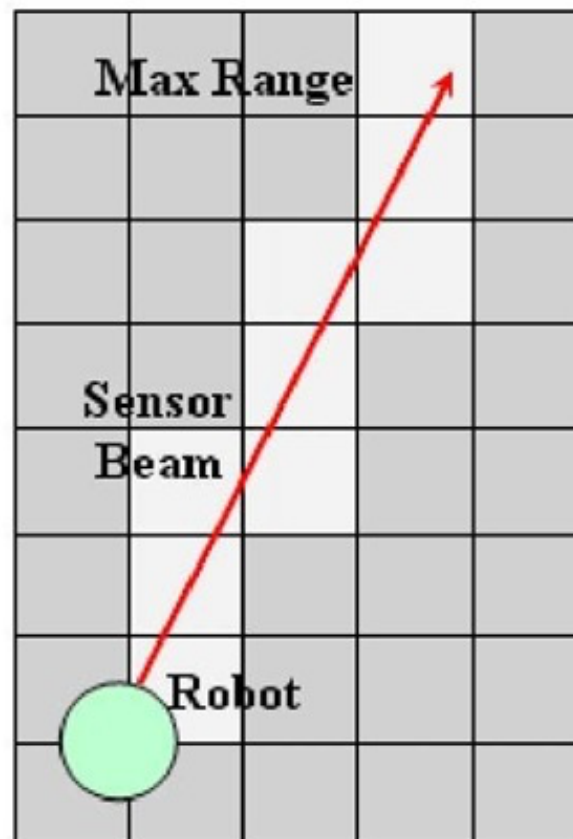
```
1:  $\mathcal{X}_{t+1} = \emptyset$ 
2:  $\eta = 0$ 
3: for  $i = 1 \dots N$  do
4:   Ziehe ein  $j$  entsprechend der Verteilung über  $w_t$  // Bel( $\mathbf{x}_t$ )
5:   Ziehe ein  $\mathbf{x}_{t+1}^i$  entsprechend  $P(\mathbf{x}_{t+1} | \mathbf{u}_{t+1}, \mathbf{x}_t^j)$  // Transition
6:    $w_{t+1}^i = P(z_{t+1} | \mathbf{x}_{t+1}^i)$  // Sensormodell
7:    $\eta = \eta + w_{t+1}^i$  // Update des Normierungsfaktors
8:    $\mathcal{X}_{t+1} = \mathcal{X}_{t+1} \cup \{\langle \mathbf{x}_{t+1}^i, w_{t+1}^i \rangle\}$ 
9: end for
10: for  $i = 1 \dots N$  do // Normierung der Gewichte in  $\mathcal{X}_{t+1}$ 
11:    $w_{t+1}^i = \eta^{-1} w_{t+1}^i$ 
12: end for
13: return  $\mathcal{X}_{t+1}$ 
```



Simulation von Scans



(a)



(b)

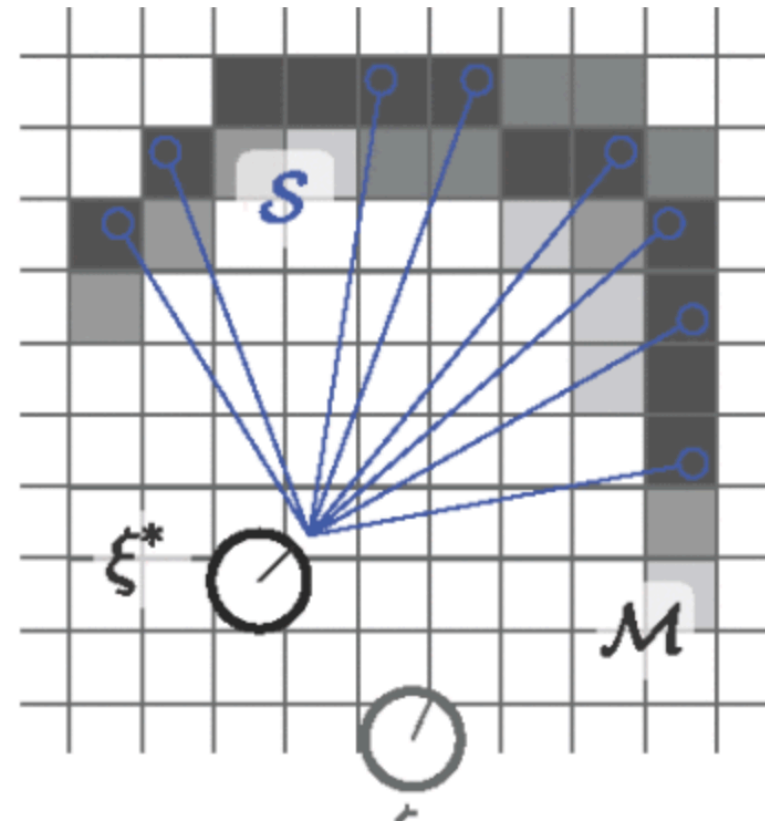
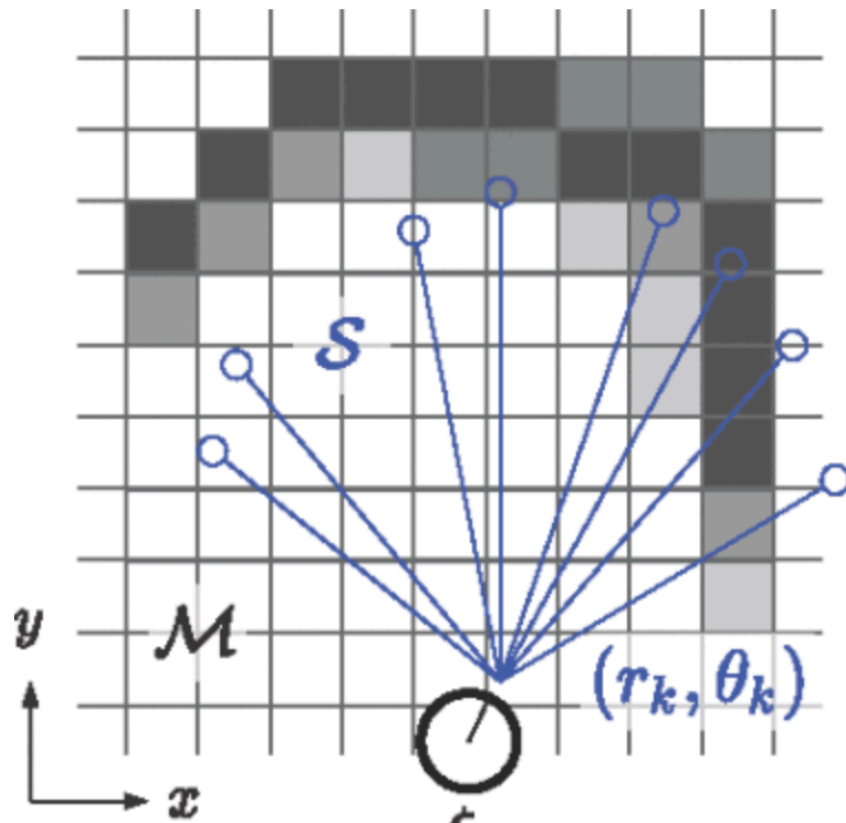


Vergleich realer vs. erwarteter Scan

- ▶ ... realisiert/approximiert Zeile 6 MCL: $w_{t+1}^i = P(z_{t+1} | x_{t+1}^i)$
- ▶ Definiere $\xi(\mathbf{X}, \mathbf{Z})$, z.B.:
- ▶ „Simuliere“ in Rasterkarte Scan von Pose von X
 - ▶ Berechne einzelne Messwerte in den diskreten Scanwinkeln des realen Scanners)
 - ▶ Ermittle Kreuzkorrelation (siehe Winkelhistogramme) zwischen realem und simuliertem Scan.
 - ▶ Summiere Messwertdifferenzen gewichtet:
 - ▶ Kleine reale Messwerte bekommen ein kleines Gewicht (nicht kartierte Möbel / Personen)
 - ▶ Große Reale Messwerte bekommen ein großes Gewicht



Vergleich Scan vs. Karte





Eingabe : Partikelmenge $\mathcal{X}_t = \{\langle \mathbf{x}_t^i, w_t^i \rangle\}_{i=1, \dots, N}$, Aktion $\mathbf{u}_{t+1}$, Messung z_{t+1} .

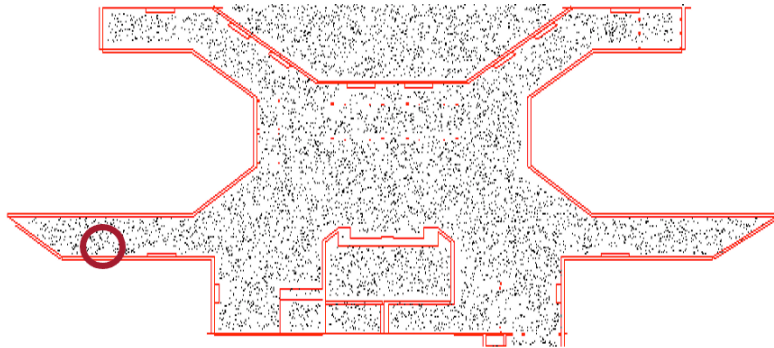
Ausgabe: Neue Menge von Partikeln $\mathcal{X}_{t+1}$

```
1:  $\mathcal{X}_{t+1} = \emptyset$ 
2:  $\eta = 0$ 
3: for  $i = 1 \dots N$  do
4:   Ziehe ein  $j$  entsprechend der Verteilung über  $w_t$  // Bel( $\mathbf{x}_t$ )
5:   Ziehe ein  $\mathbf{x}_{t+1}^i$  entsprechend  $P(\mathbf{x}_{t+1} | \mathbf{u}_{t+1}, \mathbf{x}_t^j)$  // Transition
6:    $w_{t+1}^i = P(z_{t+1} | \mathbf{x}_{t+1}^i)$  // Sensormodell
7:    $\eta = \eta + w_{t+1}^i$  // Update des Normierungsfaktors
8:    $\mathcal{X}_{t+1} = \mathcal{X}_{t+1} \cup \{\langle \mathbf{x}_{t+1}^i, w_{t+1}^i \rangle\}$ 
9: end for
10: for  $i = 1 \dots N$  do // Normierung der Gewichte in  $\mathcal{X}_{t+1}$ 
11:    $w_{t+1}^i = \eta^{-1} w_{t+1}^i$ 
12: end for
13: return  $\mathcal{X}_{t+1}$ 
```

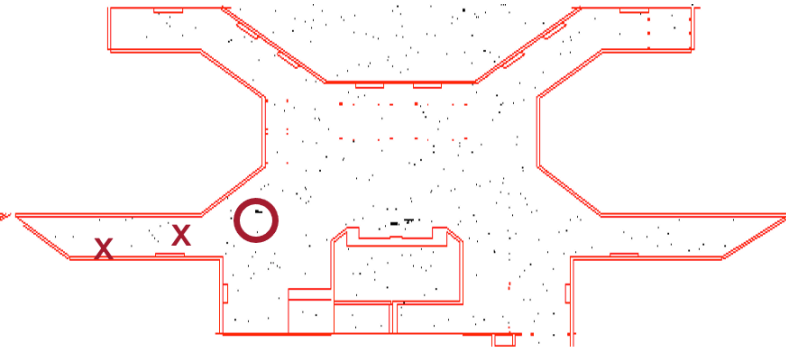


Beispiel MCL

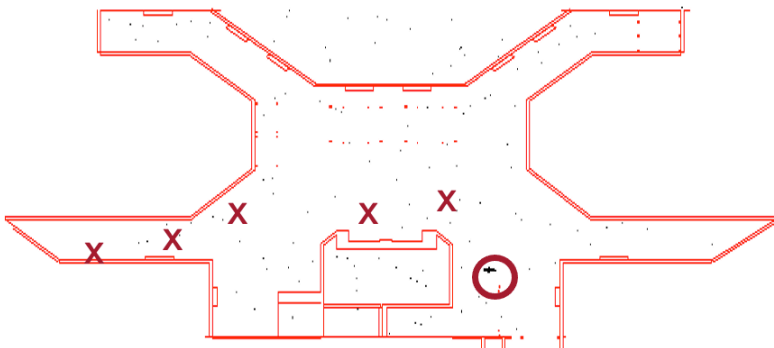
Aktuelle Position markiert mit Kreis; Wegpunkte markiert mit x



Start: Gleichverteilung im Freiraum



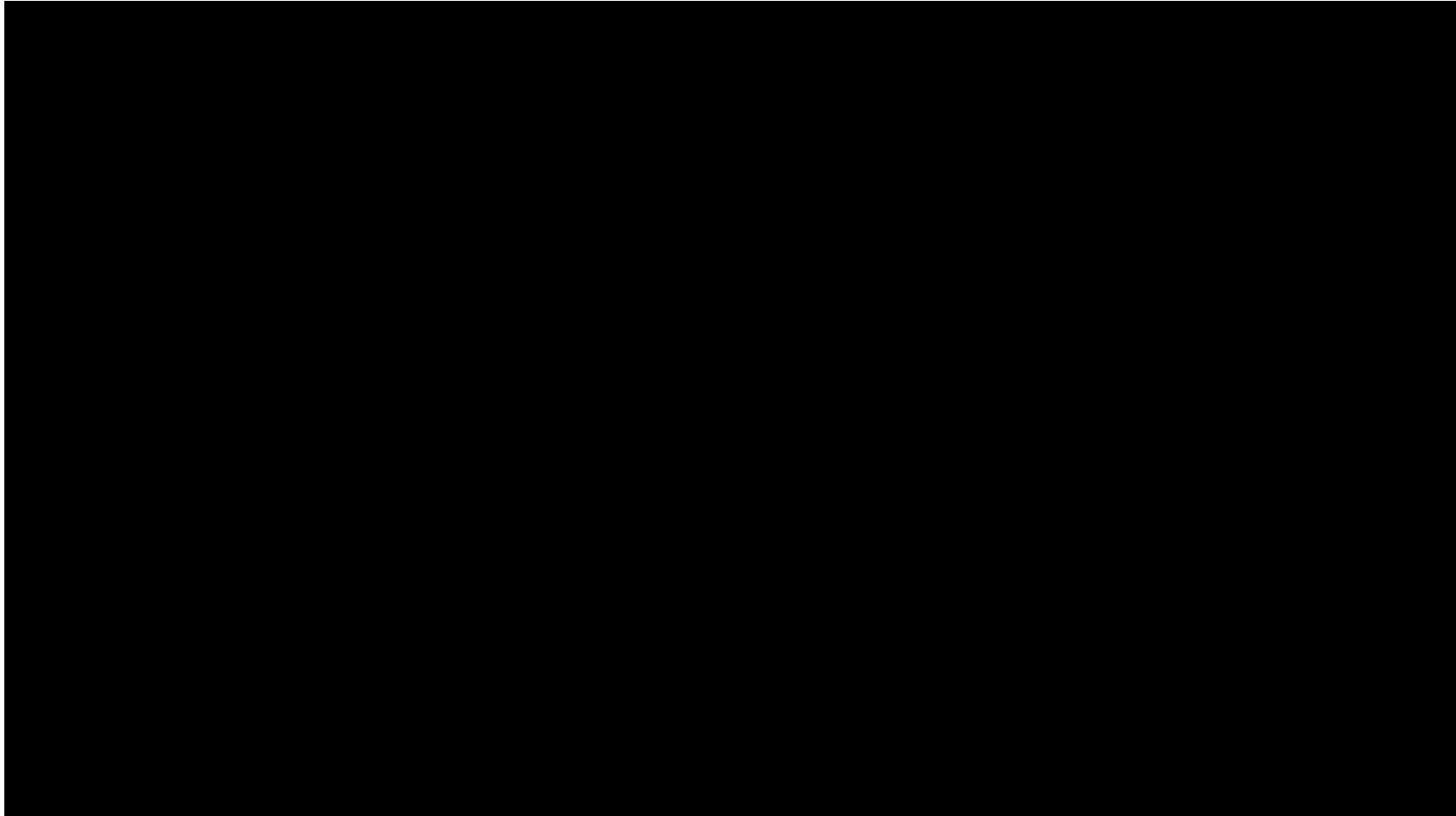
Dritte Messung: wenige Maxima



Sechste Messung: 1 Maximum

„Absterbende“ Partikel
(Untergewicht, unmögliche Pose
in der Wand, ...) ersetze:

- zufällig in Freiraum und/oder
- gewichtet nach aktueller
Verteilung



https://myshare-1.rz.uni-osnabrueck.de/seafhttp/files/d8516dc5-4585-4f81-996c-b87d6c48d54a/Adaptive_Monte_Carlo_Localization_in_ROS.mp4

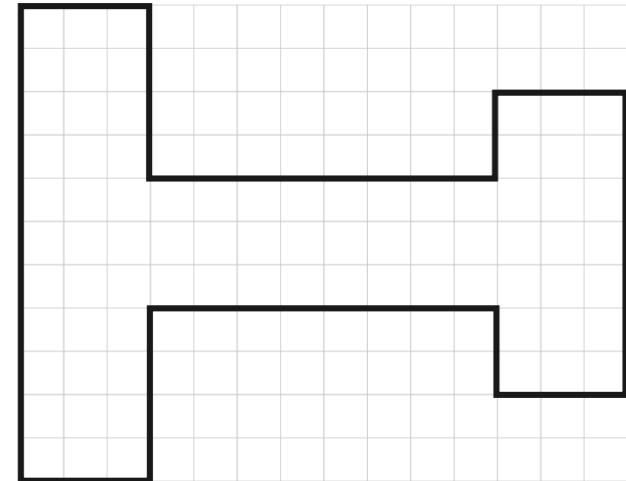


Beispielaufgabe

Spielen Sie nun die einzelnen Schritte der Monte-Carlo-Lokalisierung für die folgende Abfolge von Aktionen in der unten gezeigten Umgebung mit den vorgegebenen Messergebnissen durch. Die Karten sind kontinuierlich, die Raster dienen nur als Hilfe für die Roboterbewegung.

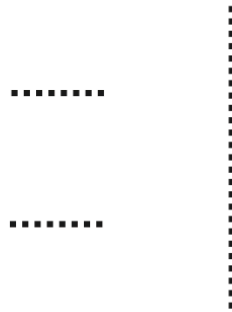
Zeichnen Sie für jede Aktion jeweils eine plausible Partikelverteilung ein. Geben Sie zu jeder Zeichnung eine kurze Erklärung.

Initialisierung

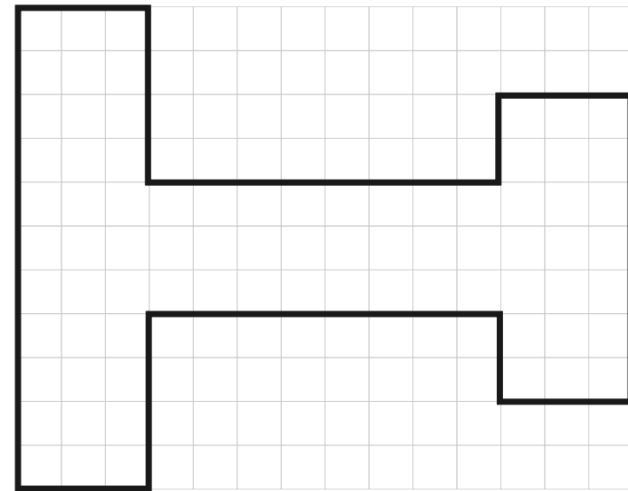


Beispielaufgabe

Zeichnen Sie für jede Aktion jeweils eine plausible Partikelverteilung ein. Geben Sie zu jeder Zeichnung eine kurze Erklärung.



Erste Messung (Messergebnis links)



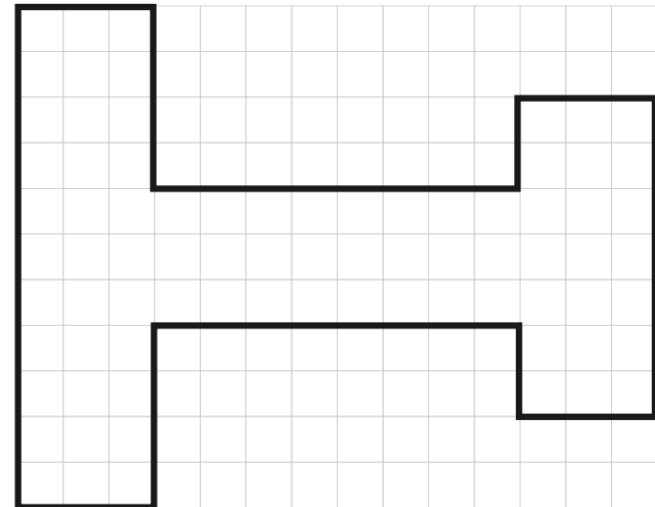


Beispielaufgabe

Spielen Sie nun die einzelnen Schritte der Monte-Carlo-Lokalisierung für die folgende Abfolge von Aktionen in der unten gezeigten Umgebung mit den vorgegebenen Messergebnissen durch. Die Karten sind kontinuierlich, die Raster dienen nur als Hilfe für die Roboterbewegung.

Zeichnen Sie für jede Aktion jeweils eine plausible Partikelverteilung ein. Geben Sie zu jeder Zeichnung eine kurze Erklärung.

Bewegung um 2 Einheiten nach vorne



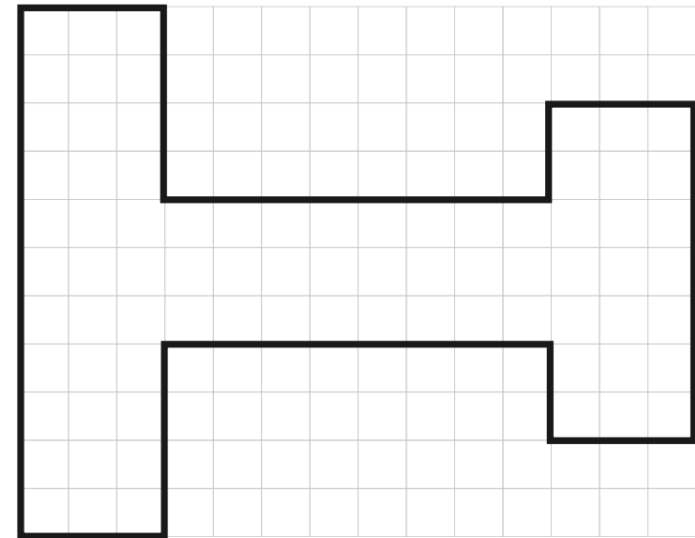
Beispielaufgabe

Spiele Sie nun die einzelnen Schritte der Monte-Carlo-Lokalisierung für die folgende Abfolge von Aktionen in der unten gezeigten Umgebung mit den vorgegebenen Messergebnissen durch. Die Karten sind kontinuierlich, die Raster dienen nur als Hilfe für die Roboterbewegung.

Zeichnen Sie für jede Aktion jeweils eine plausible Partikelverteilung ein. Geben Sie zu jeder Zeichnung eine kurze Erklärung.



Zweite Messung (Messergebnis links)





MCL ist ein Bayes-Filter

$$Bel(x_t) = \eta p(z_t | x_t) \int p(x_t | x_{t-1}, u_t) Bel(x_{t-1}) dx_{t-1}$$

Diagram illustrating the components of the equation:

- η : Importance Factor für x_t^i
- $p(z_t | x_t)$: Ziehe x_t^i aus $p(x_t | x_{t-1}^i, u_t)$
- $Bel(x_{t-1})$: Ziehe x_{t-1}^i aus $Bel(x_{t-1})$



Meint der Roboter, dass er weiß wo er ist?

Erinnerung: **Entropie** einer Verteilung $P(X)$ über Var.n X :

$$H(X) := - \sum_{i=1}^{|X|} P(x_i) \log_2 P(x_i)$$

... ist maximal bei Gleichverteilung, nämlich

$$H_{\max}(X) = - \sum_{i=1}^{|X|} \frac{1}{|X|} \log_2 \frac{1}{|X|} = -|X| \cdot \frac{1}{|X|} (\log_2 1 - \log_2 |X|) = \log_2 |X|$$

(Lokalisation: Posen gleichverteilt, Roboter hat keine Idee, wo er ist)

... ist minimal bei sicherer Information, nämlich 0 (wegen $\log(1)=0$)

$$H(\mathcal{X}) := - \sum_{\mathbf{x} \in \mathcal{X}} Bel(\mathbf{x}) \frac{\log_2 Bel(\mathbf{x})}{\log_2 |\mathcal{X}|} = - \sum_{\mathbf{x} \in \mathcal{X}} Bel(\mathbf{x}) \log_{|\mathcal{X}|} Bel(\mathbf{x})$$

...normiert auf $[0,1]$ für Belief bezüglich Partikelmenge $\mathcal{X}$

Unsicherheitsmaß des aktuellen Beliefs $Bel(\mathcal{X})$

($H(\mathcal{X})=0 \rightarrow$ genaue Kenntnis der Pose; $H(\mathcal{X})=1 \rightarrow$ völlige Unkenntnis)



- ▶ Statistischer Ansatz, um die Anzahl der Samples während der Laufzeit zu adaptieren
- ▶ Gute Approximation der gewünschten Verteilung $\Rightarrow$ wenige Samples
- ▶ Berechne “Abstand” zwischen anvisierter Verteilung (a-posteriori) und der durch die Samples repräsentierten Verteilung
- ▶ Fehlermaß: Kullback-Leibler-Divergenz:

$$K(P, P') = \sum_{i=1}^{|\mathcal{X}|} P(x^i) \log \frac{P(x^i)}{P'(x^i)}$$

- ▶ Ziehe weitere Stichproben, bis der Abstand K kleiner ist als vorgegebene Schranke



Wie viele Stichproben?

- ▶ Kommt drauf an! ...
 - ▶ ...wie groß ist relevante Posemenge: klein $\Rightarrow N$ klein; groß $\Rightarrow N$ groß
 - ▶ ...wie häufig Landmarken in der Umgebung: viele $\Rightarrow N$ klein; wenige $\Rightarrow N$ groß
 - ▶ ...wie gut Pose aktuell bekannt (variabel!): $H(\chi)$ nahe 0 $\Rightarrow N$ klein; $H(\chi)$ nahe 1 $\Rightarrow N$ groß
- ▶ Für Gebäudenavigation reicht $N \ll 10.000$
- ▶ Für kleine Areale (RoboCup!) reicht $N \ll 1.000$
- ▶ KLD-Sampling adaptiert N



Landmarken?



"Face it, Fred—you're lost!"



Fazit Lokalisierung in Karten

- ▶ Entsprechend Varianten von Karten und Varianten von Sensoren gibt es viele Varianten von Lokalisierung in Karten
- ▶ Markow-Lokalisierung mit Monte-Carlo-Sampling (MCL) dominiert derzeit Literatur
 - ▶ Mathematisch gut verstanden; effizient implementierbar
 - ▶ Integration mit anderer probabilistischer Information
 - ▶ Integration mit Kartenbau (SLAM, s.u.)
- ▶ Fast alle Karten / Lokalisierung sind für 2D-Roboterbewegung (x, z, θ)
- ▶ Verwenden fast nur Geometrie, keine „Semantik“
- ▶ Klappen nur bei relativ langsamer Fahrt (Ausnahme HAYAI)
- ▶ Probleme bei Umgebungsdynamik
- ▶ ➡ Es gibt nach wie vor einiges zu erforschen!

